Big Data and NoSQL - BookScope

ANDRÉ Mathis
VIALE Jean-Jacques

Introduction

Une bibliothèque numérique veut aider ses lecteurs à découvrir des livres autrement que par une recherche par mots-clés ou par le classement des titres les plus populaires. BookScope s'appuie sur le catalogue (titres, auteurs, tags) et sur l'historique des lecteurs (notes, listes de lecture). La plateforme permet d'explorer les liens entre livres, lecteurs et thèmes, et de recommander des livres non encore notés. Chaque résultat peut être relié aux enregistrements qui le justifient. Les livres très populaires et les lecteurs très actifs ne doivent pas masquer les intérêts plus rares. Nous travaillons sur une sélection de plus d'un million de notes réelles du jeu de données Goodbooks-10k.

Les données sont stockées dans MongoDB 9.0.4, qui tourne comme processus serveur séparé. Les six services sont des scripts Python en ligne de commande, qui interrogent la base avec pymongo. Les entrées sont fournies à l'exécution, et chaque réponse est calculée à partir des données stockées.

La même interface sert à deux déploiements : un serveur local, et un cluster de deux shards placé derrière un routeur mongos. On passe de l'un à l'autre en changeant une variable d'environnement, sans modifier le code. Toute la chaîne, de la préparation et de l'import jusqu'aux mesures, est reproductible à partir des fichiers CSV d'origine.

Toutes les mesures sont faites sur un seul poste : Ryzen 7 9800X3D, 31 Go de RAM, Windows 11. Les processus du cluster partagent donc le même matériel, et ce déploiement ne vaut pas une grappe de machines indépendantes.

Données

Une règle de sélection a été déterminée afin de réduire la quantité de données, seules les informations liées à des utilisateurs ayant un id entre 20000 et 29999 seront gardées. Cette sélection nous donne 1 128 761 notes réelles données par 10 000 lecteurs. Prendre une plage d’identifiant permet de sélectionner aléatoirement les utilisateurs en évitant de favoriser un thème.
Toutes les informations propres aux livres et non liées aux utilisateurs restent dans la base de données : 10 000 livres, 34 252 tags, 999 904 associations livre–tag. 2 513 livres ont moins de 20 notes dans la base. Pour eux, les liens trouvés par F1 et F4 sont faibles.

Ce set de données a ensuite suivi un découpage en 3 ensembles disjoints: 80% du set est consacré à la base initiale (ce qui fait 903 033 notes). Ensuite le lot de mis à jour importé par F6 et celui dédié à l’évaluation de E3 comprendront chacun 10%.

La répartition de l’activité chez les lecteurs est régulière : 90 notes en médiane (la valeur du milieu), entre 18 et 159, et 80 % des lecteurs ont entre 66 et 117 notes.
Elle est en revanche très concentrée pour les livres : 35 notes en médiane, mais 3 998 pour le livre le plus noté (The Hunger Games). Les 100 livres les plus notés concentrent 18,8 % des notes, et les 10 % de livres les plus notés en concentrent 56,3 %. C'est ce biais de popularité que les services doivent compenser pour ne pas toujours proposer les mêmes best-sellers.On considère donc qu'un lecteur aime un livre à partir d'une note de 4.
Aucun lecteur n'a moins de 18 notes. Pour simuler un nouveau lecteur, on tronque l'historique d'un lecteur réel en ne gardant que ses k premières notes, avec l'option --truncate k.

Modélisation et Prototype local

Le prototype local utilise une seule instance mongod, que les scripts interrogent via localhost:27017. Il y a une collection par fichier source : ratings, books, book_tags, tags et to_read (182 725 entrées). Chaque note est un document {user_id, book_id, rating}. Son identité est le couple (user_id, book_id), qu'un index unique protège contre les doublons lors des imports F6. Les notes et les listes de lecture désignent un livre par son book_id, alors que book_tags utilise le goodreads_book_id. La collection books, qui porte les deux identifiants, sert de pont entre les deux.

Les services reposent sur trois accès. “Qui a aimé ce livre ?” et “Qu'ont aimé ces lecteurs ?” servent à F1, F4 et F5. “Quelles sont les notes de ce lecteur ?” sert à F2 et F3. Des index de ratings qui commencent par book_id ou par user_id couvrent ces accès, et le catalogue est indexé sur ses identifiants. Nous avons vérifié les plans d'exécution avec explain.

Comme alternative, nous avons implémenté et mesuré des index couvrants, (book_id, rating, user_id) et (user_id, rating, book_id). Ils contiennent tous les champs demandés, et MongoDB répond donc sans lire les documents. Le script bench.py compare les deux versions sur les mêmes entrées, avec un passage à froid puis la médiane de 5 passages à chaud. Les résultats sont identiques, seul le temps change pour « livres aimés par les lecteurs du livre 1 », on passe de 308 065 documents lus en 236 ms à aucun document lu en 70 ms. Ainsi que sur les services complets, le temps baisse de 45 % pour F1, 39 % pour F3, 61 % pour F4 et 63 % pour F5 (écart-type ≤ 0,15 s). F2 ne change pas, son temps est passé surtout dans la jointure avec les tags.
En contrepartie, les index de ratings passent de 34 à 54 Mo, et l'import du lot F6 passe de 0,88 s à 1,19 s. Nous avons gardé les index couvrants, car les lectures sont interactives alors que les écritures se font par lots.

Nous avons écarté deux autres modèles sans les implémenter. Un document par lecteur favoriserait F2 et F3, mais pénaliserait F1, F4 et F5, qui partent d'un livre. Des liens précalculés entre livres devraient être recalculés à chaque import F6. Nous calculons donc tous les résultats à la demande.

La base se reconstruit et se remesure avec prepare_data.py, import_data.py (qui vide la base avant l'import) et bench.py. Le découpage ne fait appel à aucun hasard, et les versions des bibliothèques sont figées dans requirements.txt.

Services

F1 - Explorer un livre et ses connections

F1 prend en entrée un identifiant de livre, ou un titre. Il renvoie les informations du livre (auteur, année, note, moyenne, série, tags), ainsi qu'une liste de livre triés, classés en fonction de leur score accompagnés des preuves.
Pour les recherches par titre, il y a d'abord une vérification qui va être effectuée. On recherche des similarités avec les titres et les titres originaux, si il y a plusieurs oeuvres correspondantes, alors F1 renvoie la liste de celles- ci, en comprenant leurs identifiants.

L'évaluation se base sur deux scores, un basé sur les tags et l'autre sur les lecteurs.
Le score des thèmes compare les tags de deux livres. Pour ça les tags sont pondérés par leur rareté afin d'éviter que les tags trop courants faussent la similarité (comme "to_read" possédé par 99.8% des livres). Cela permet aussi de donner plus d'importance aux tags rares. Le score thématique est la part du poids des tags du livre de départ qu'on retrouve dans l'autre livre, entre 0 et 1. Un ratio est d'ailleurs fixé pour que les tags portés par plus de 50% des livres ne soient pas pris en compte.
Le score des lecteurs se base sur les lecteurs ayant aimé les deux livres. Le problème dans cette approche est que la présence de "best seller" et de gros lecteurs peuvent dominer le reste. Pour empêcher cela, un lecteur qui aime beaucoup de livres aura moins d'impact qu'un autre lecteur qui serait plus sélectif. Le nombre de lecteurs en commun est rapporté à la popularité des deux livres via la similarité cosinus (vaut 1 si deux livres ont exactement les mêmes lecteurs, et 0 s' ils en ont aucun en commun). Deux livres doivent avoir au moins 3 lecteurs en commun pour que leur lien soit retenu.

F1 affiche aussi le lift, qui indique si les lecteurs communs à deux livres révèlent un vrai lien ou seulement leur popularité. On le calcule en comparant le nombre de lecteurs ayant aimé les deux livres au nombre attendu si aimer l'un n'avait aucun effet sur le fait d'aimer l'autre. Ce nombre attendu se déduit de la popularité de chaque livre : si 10 % des lecteurs aiment A, on s'attend à ce que 10 % des lecteurs de B l'aiment aussi par hasard. Concrètement, lift = (lecteurs communs × nombre total de lecteurs) / (lecteurs de A × lecteurs de B). Un lift de 2 signifie que les lecteurs de B aiment A deux fois plus souvent que la moyenne des lecteurs : les deux livres attirent le même public. Un lift proche de 1 signifie que ces lecteurs communs s'expliquent seulement par la popularité des deux livres. Cela évite de relier deux best-sellers qui n'ont rien à voir.

Le score final est donc constitué de la moyenne de ces deux scores. Auxquels on ajoute deux possibles bonus, +0.3 si un auteur est le même, et +0.1 si l'année de publication est proche. En cas d'égalité les livres sont triés par id croissant.
Néanmoins les livres d'une même série sont forcément ce qu'il y a de plus similaire, alors ils sont automatiquement placés en tête de file. Ils sont récupérés via un pattern regex.
Chaque lien est enfin qualifié selon sa nature : même série, même auteur, lectorat partagé (au moins 3 lecteurs communs et un lift supérieur à 1), thème spécifique (au moins un tag commun qui n'est pas très fréquent), ou lien dû uniquement à des tags très fréquents. Cette dernière catégorie permet de distinguer, comme le demande le sujet, un vrai lectorat commun d'un lien créé seulement par un tag que presque tous les livres portent.

Par exemple, pour The Hunger Games (livre 1), aimé par 3 360 lecteurs de la base, F1 propose d'abord Catching Fire (livre 17) : même série, même auteur, 1 516 lecteurs communs, soit 2,3 fois plus que le hasard, et 68 tags spécifiques communs. Arrivent ensuite Mockingjay et le coffret de la trilogie, puis Divergent (livre 12), d'un autre auteur mais avec 765 lecteurs communs (×1,8) et des tags précis comme « ya-dystopian ». Harry Potter à l'école des sorciers (livre 2) apparaît en cinquième position. Son score thématique est faible (0,27), parce que ses tags communs avec le livre 1 sont surtout génériques (« favourite-books », « read-more-than-once »), mais son lectorat commun est fort (1 860 lecteurs, ×1,7). F1 sépare ainsi clairement un lien par le thème d'un lien par les lecteurs. Pour vérifier un lien, l'option --trace liste tous les enregistrements qui le justifient : chaque lecteur commun avec ses deux notes, et chaque tag commun avec son nombre d'occurrences sur chacun des deux livres (python f1.py 1 --trace 2).

F2 - Trouver les similarités entre deux utilisateurs

F2 prend en entrée deux identifiants de lecteurs et renvoie un score de similarité basé sur les notes des livres en commun et les genres de livres appréciés par les deux utilisateurs.

Le score de similarité entre les livres communs est calculé à partir de l'écart entre les notes. Si les deux utilisateurs ont mis la même note, le score sera de 100% alors qu'un écart de 4 donnera un score de 0%.

Écart de note	**Similarité**
0	100%
1	75%
2	50%
3	25%
4	0%

À la fin, on fait la moyenne de ces scores pour avoir un pourcentage basé sur l'ensemble des livres en commun.

Pour le score de similarité basé sur les genres de livres aimés par les deux utilisateurs, on utilise une liste de genre tels que "fantasy", "romance", "horror" et plus qui sont définis dans config.py. Cette liste a été définie à partir des tags intéressants les plus populaires.
On regarde ensuite parmi tous les livres notés par chacun des utilisateurs leurs tags. Si le tag contient un mot-clé qui appartient à un genre, on incrémente ou décrémente le score du genre pour l'utilisateur en fonction de la note attribuée au livre, le calcul est : (note - 3) / 2.
Ensuite on utilise la similarité cosinus, cela nous permet de mesurer la proximité de deux vecteurs à partir de leur orientation, ce qui permet de comparer les préférences des utilisateurs sans prendre en compte le nombre de notes des utilisateurs.

Une fois que l'on a ces deux scores, on calcule la similarité globale en les additionnant avec une pondération. Nous avons choisi 60% pour les livres communs et 40% pour les genres.

F3 - Recommander un livre à un lecteur

F3 prend en entrée un identifiant de lecteur. Il va ensuite renvoyer une liste de livres recommandés pour le lecteurs (qu'il n'a pas encore lus).

La recommandation des livres va se baser sur F1. On commence par sélectionner 10 livres que l'utilisateur a appréciés (les livres graines), puis appliquer F1 sur chacun d'eux. 10 livres vont donc être récupérés pour chacun de ces livres graines. Chaque apparition de livre dans les résultats F1 va faire gagner un point au livre candidat, sa similarité à plusieurs livres appréciés par l'utilisateur va donc le faire monter dans le classement. En cas d'égalité, les livres sont départagés par la sommes des scores F1, puis par identifiant croissant.
Chaque recommandation affiche les livres graines auxquels il est lié, ainsi que la raison donnée par F1. L'utilisation de F1 permet de limiter l'impact des best-seller et d'empêcher de simplement reproduire un classement de popularité global.

Par exemple, pour le lecteur 20000 (91 notes), les 10 graines sont toutes notées 5 : Unbroken, The Handmaid's Tale, The Brief Wondrous Life of Oscar Wao, etc. F3 recommande d'abord Wild, proposé par trois graines, dont Unbroken (78 lecteurs communs, ×4,9). Viennent ensuite Where'd You Go, Bernadette et The Interestings, eux aussi proposés par trois graines. Drown arrive en quatrième position parce que Junot Díaz est l'auteur de deux graines. Aucune des 10 recommandations ne fait partie des 100 livres les plus notés. Leur popularité médiane est de 236 notes, contre 2 606 pour une recommandation par popularité, soit environ 11 fois moins.

F4 - Tracer une connexion entre des livres

F4 prend en entrée deux livres (via identifiant ou nom comme le fait F1). Il renvoie les chemins qui relient les deux livres, classés du plus au moins convaincants, suivis par leurs preuves.
Un chemin est une suite de livres ayant un lectorat partagé à chaque étape. Une étape est convaincante si au moins 3 lecteurs ont aimés les deux et que leur lift vaut au moins 1.5.
F4 cherche d'abord un lien direct, puis les chemins à 2 étapes (A → X → B), où X est voisin des deux livres. S'il manque des chemins, il cherche aussi des chemins à 3 étapes (A → X → Y → B) à partir des 20 voisins les plus forts de A.
Afin d'éviter des chemins inintéressants, les 100 livres les plus aimés, ainsi que le top 5% des lecteurs ayant le plus aimé de livres sont ignorés.
Chaque chemin est ordonné par premièrement son nombre d'étapes, puis par sa force. La force d'un chemin est celle de son étape la plus faible et est déterminée par sa similarité cosinus.
Dans le cas où aucun chemin pertinent n'est trouvé, la justification est tout de même affichée (aucun voisin convaincant, passage obligatoire par un carrefour etc.).

Par exemple, entre The Hunger Games (livre 1) et Passion Unleashed (livre 5000, romance paranormale), F4 trouve 29 chemins à 2 étapes. Le meilleur passe par Darkfever (livre

1070). Sa première étape repose sur 63 lecteurs communs (×1,5). La seconde repose sur 6 lecteurs (×13,9), et les tags communs « paranormal-urban-fantasy » et « romance-paranormal » viennent en contexte. Le chemin est donc qualifié de moyen. Les alternatives passent par Frostbite (Vampire Academy) et Untamed (House of Night), des séries young adult paranormales, ce qui est cohérent. Un autre chemin a été écarté parce qu'il passait par un carrefour.

Entre Pretend You Don't See Her (livre 9000, thriller) et The Kings of Clonmel (livre 4000, fantasy jeunesse), il ne reste que des chemins à 3 étapes, tous fragiles. Les trois premiers reposent sur les trois mêmes lecteurs. Un lift très élevé (×633) sur 3 lecteurs ne prouve donc pas grand-chose. Enfin, entre The History of the Hobbit (livre 1129) et le livre 1, F4 ne trouve aucune connexion. Il explique que le livre 1129 n'a aucun voisin convaincant, car un seul lecteur de la sélection l'a aimé.

F5 - Découvrir des liens entre intérêts

F5 prend en entrée deux tags (par identifiant ou nom exact). Et il renvoie une évaluation du lien entre ces deux thèmes avec un groupe de lecteurs qui aiment les deux, et une liste de livres pont.
Un lien est formé via les lecteurs et non les tags directement. Un lecteur appartient au groupe d'un tag s'il aime au moins 3 livres qui le porte, et plus que la moyenne des lecteurs. Le lien entre lecteurs se fait donc au recoupement de leurs deux groupes, mesuré par le lift : le nombre de lecteurs présents dans les deux groupes, comparé au nombre attendu si les deux étaient indépendants.
Un livre pont est un livre aimé par au moins 3 lecteurs de chaque groupe. Il doit aussi être aimé plus souvent dans chacun des deux groupes que dans l'ensemble des lecteurs. Les livres pont sont classés par la plus petite des deux parts (la part du groupe 1 qui l'a aimé, la part du groupe 2 qui l'a aimé), car un pont doit tenir des deux côtés. Pour chaque livre pont, F5 affiche le nombre de lecteurs de chaque groupe qui l'ont aimé, le nombre de fois où chaque tag a été posé sur ce livre, et quelques lecteurs en exemple avec leur note.
Afin de vérifier l'intérêt d'un lien, le calcul est refait sans les 100 livres les plus lus et sans le top 5% des lecteurs les plus actifs. Ce qui permet de définir le cas représenté soit ils n'ont pas de lien (moins de 3 lecteurs communs), un lien faible (lift inférieur à 1,5), le lien tient sans les livres populaires ni les lecteurs actifs, le lien n'existe plus sans eux.
Par exemple, entre fantasy et romance, 2 007 livres portent les deux tags, ce qui pourrait laisser croire à un lien fort. Pourtant, le groupe fantasy (4 503 lecteurs) et le groupe romance (4 170 lecteurs) partagent 2 276 lecteurs, soit seulement ×1,21 par rapport au hasard. F5 conclut donc à un lien faible. Sans les livres populaires ni les lecteurs actifs, le lift reste à ×1,27. Sans les livres à double étiquette, il tombe à ×0,59, donc sous le hasard : le peu de lien qui existe vient de ces livres, et non de lecteurs qui aimeraient les deux genres séparément. Les livres pont bruts sont des best-sellers (The Hunger Games, quatre tomes de Harry Potter). Une fois ceux-ci retirés, ce sont City of Bones, City of Glass, City of Ashes, Eragon et The Last Olympian. Aucun des cinq livres pont bruts ne reste dans le top 5 : ils n'étaient là que grâce à leur popularité.

Prototype distribué

Le cluster reprend l'architecture du Lab 5. Il comprend un serveur de configuration (port 27200), deux shards shard1RS et shard2RS (ports 27301 et 27302, chacun avec son propre dossier de données) et un routeur mongos (port 27400), seul point d'entrée des scripts. Chaque shard est un replica set à un seul membre donc les données ne sont donc pas copiées. Le script cluster.py démarre, arrête et décrit le cluster.
Les notes et les listes de lecture sont réparties selon la clé *{user_id: "hashed"}*. Le catalogue (books, book_tags, tags) n'est pas réparti et reste sur le shard primaire. Nous avons choisi user_id parce que F2, F3, F6 et E3 partent d'un lecteur : toutes ses notes se trouvent alors sur un seul shard. De plus, l'index unique (user_id, book_id), qui définit l'identité d'une note, commence bien par la clé de répartition, comme MongoDB l'exige. Le hachage répartit les lecteurs uniformément, avec 445 160 notes sur shard1RS et 457 873 sur shard2RS. En contrepartie, toute requête centrée sur un livre doit interroger les deux shards.
On en ensuite mesuré l'effet sur trois types de requêtes (explain via mongos, script dist_explain.py). Une requête centrée lecteur, comme les notes du lecteur 20000, ne contacte qu'un seul shard : celui-ci lit 91 clés d'index et renvoie les 91 notes à mongos, en 1 ms. Une requête centrée livre, comme les lecteurs ayant aimé le livre 1, interroge les deux shards : chacun lit sa partie de l'index (1 630 et 1 730 clés), puis mongos fusionne les deux listes, en 5 ms. Le parcours livre 1 → ses lecteurs → les livres qu'ils ont aimés est le plus coûteux. mongos envoie à chaque shard la liste des 3 360 lecteurs (32 Ko). Chaque shard compte, parmi ses propres lecteurs, combien ont aimé chaque livre, et renvoie ces comptes partiels (8 582 et 8 704). Comme un shard ne connaît qu'une partie des lecteurs d'un livre, mongos doit additionner ces comptes par un second $group. L'ensemble prend 156 ms.
Le script compare.py lance F1 à F5 avec les mêmes entrées sur le serveur local et sur les deux placements du cluster. Il compare ensuite les sorties sous forme canonique : nombres réels arrondis à 10^{-9}, car un $group réparti additionne dans un autre ordre, et listes triées par score puis par identifiant. Les entrées couvrent des lecteurs et des livres situés sur des shards différents, par exemple F2 sur les lecteurs 20000 (shard2RS) et 22534 (shard1RS). Résultat : 26 comparaisons sur 26 sont identiques.

Configuration alternative : placement par livre (*{book_id: "hashed"}*). Les accès s'inversent : les lecteurs d'un livre se trouvent sur un seul shard, alors que les notes d'un lecteur sont réparties sur les deux. Dans le parcours, chaque shard renvoie des comptes complets que mongos se contente de concaténer (136 ms au lieu de 156 ms), ce qui accélère F1 et F3 (voir E2). Nous gardons pourtant le placement par lecteur. Il est le seul à garder les requêtes personnalisées sur une seule instance, et en cas de panne, la moitié des lecteurs conserve son historique complet. En placement par livre, l'historique de tous les lecteurs serait amputé.

Une fois shard2RS arrêté, MongoDB renvoie une erreur brute au bout d'environ 15 s. Avec l'option *allowPartialResults*, il renverrait même 1 630 lecteurs au lieu de 3 360 pour le livre 1, sans aucun avertissement. Pour l'éviter, F2, F3 et F6 vérifient d'abord auprès du serveur de configuration que les shards dont ils ont besoin répondent (availability.py). Sinon, ils échouent explicitement et n'affichent aucun résultat. Ainsi, F2 refuse la paire 20000 / 22534 mais répond normalement pour 22534 / 29999, deux lecteurs stockés sur shard1RS. F3 échoue toujours, car F1 a besoin des notes de tous les lecteurs. F6 n'écrit rien plutôt que

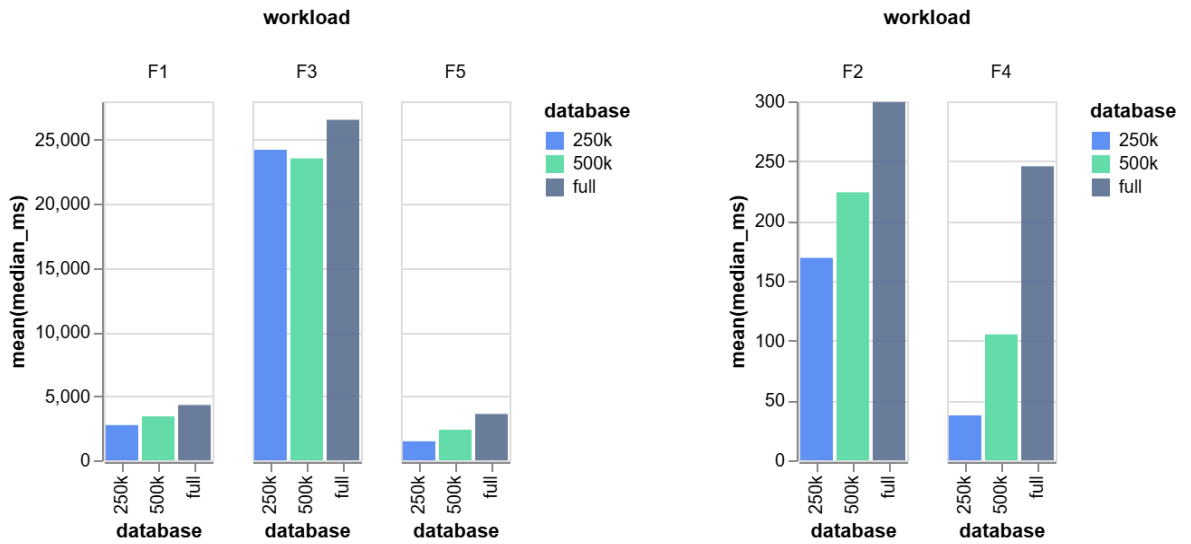
d'appliquer la moitié du lot. Notre déploiement ne récupère rien pendant la panne : les données reviennent au redémarrage du shard, mais seraient perdues avec son disque. En production, chaque shard serait un replica set de trois machines, qui élirait un nouveau primaire en quelques secondes. Ici, arrêter un processus ne simule pas une vraie panne de machine ou de réseau.

Expérience (E1-3)

E1 - Subsets de tailles différentes

Cette partie compare des métriques pour trois subsets qui ont différents nombres de notes, 250 000, 500 000, et à l'ensemble complet de taille 903 033. Chaque ensemble est compris dans le suivant, c'est-à-dire que les 250 000 de l'ensemble 1 sont aussi compris dans l'ensemble 2. Les ensembles 1 et 2 sont pris au hasard, mais une seed est utilisée pour reproduire l'expérience.

Donnée	**Ensemble 1**	**Ensemble 2**	**Ensemble 3**
Nombre de notes	250 000	500 000	903 033
Nombre de lecteurs	10 000	10 000	10 000
Nombre de livres	9 841	9 966	9 983
Taille stockée (Mo)	70,5	88,2	126,9
Temps d'importation (s)	23,76	29,97	38,71



Chaque service F1 à F5 est exécuté 5 fois pour avoir des résultats plus précis.
On peut voir que le nombre de lecteurs reste le même, le nombre de livres proche du maximum de 10 000 mais augmente quand même de 9 841 à 9 983.

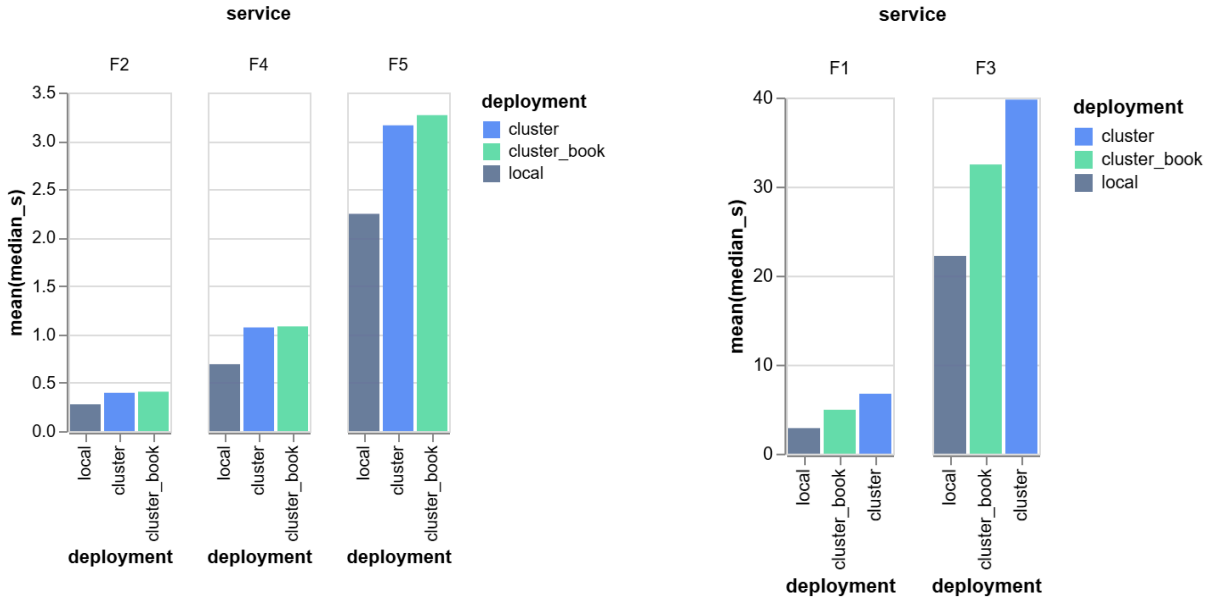
Globalement, l'augmentation du nombre d'évaluations augmente l'espace de stockage et les temps d'exécution des fonctions. F3 est de loin la fonction la plus coûteuse, mais la taille de l'ensemble ne l'affecte pas beaucoup, F2 et F4 ont des temps d'exécution très faibles.

E2 - Prototype distribué versus local

L'objectif ici est de comparer l'impact de la distribution des données pour les services F1 à F5. Trois configurations sont comparées :

- Local : une base MongoDB exécutée sans partitionnement distribué.
- Cluster par lecteur : les évaluations sont réparties sur deux shards à partir de la clé user_id.
- Cluster par livre : les évaluations sont réparties sur deux shards à partir de la clé book_id.

Les mêmes données et les mêmes entrées de test sont utilisées dans les trois configurations pour que les résultats soient comparables.
Les temps d'exécution des fonctions F1 à F5 sont mesurés 5 fois pour chaque cas de test, puis comparés par les médianes.



Les résultats montrent que la configuration local a des temps d'exécution plus faibles. Les configurations distribuées sont plus lentes, potentiellement dues à des communications supplémentaires.
La différence entre les stratégies de partitionnement montre que le choix de la clé de distribution impacte les performances.

Les résultats entre les configurations ont également été vérifiés pour s'assurer d'obtenir le même résultat peu importe la configuration.

E3 - Pertinence des recommandations

Cette expérience permet de mesurer la pertinence des recommandations de F3 en comparant à une méthode de référence basée sur la popularité des livres.

Ces métriques sont testés sur l'historique complet des utilisateurs ainsi que sur un historique des 5 premières notes des utilisateurs. L'échantillon est seulement de 20 utilisateurs par manque de temps, car la fonction F3 a un temps d'exécution très élevé.

Historique utilisateur	Méthode	Recall@K	Precision@K
Complet	BookScope	0,0896	0,090
Complet	Popularité	0,0267	0,010
Court (5 notes)	BookScope	0,0235	0,015
Court (5 notes)	Popularité	0,0267	0,010

La métrique Recall@K, mesure la proportion de livres pertinents retrouvés parmi les recommandations, et la Precision@K, mesure la proportion de recommandations qui sont pertinentes.

On peut voir que pour un historique complet, notre méthode BookScope est plus efficace que la popularité, mais qu'un historique trop court ne suffit pas à recommander des livres pertinents.

Limites et Conclusion

Nos mesures ont plusieurs limites. Elles sont faites sur un seul poste dont les shards partagent le matériel, et chaque shard n'a qu'un membre, donc aucune réplication. F3 est lent parce qu'il appelle F1 dix fois. C'est pour cette raison que E3 ne porte que sur 20 lecteurs, ce qui limite la portée de ses résultats. Avec un historique court, F3 se rapproche des best-sellers. F2 ne liste ni les livres qui justifient la comparaison, ni les désaccords. Les seuils (lift de 1,5, 3 lecteurs, 100 carrefours, 5 % de lecteurs actifs) ont été fixés à la main. Enfin, les tags et les moyennes Goodreads, calculés sur tous les lecteurs, peuvent laisser fuiter de l'information dans E3.

BookScope remplit néanmoins ses objectifs. Les six services fonctionnent sur les deux déploiements et donnent des résultats identiques (26 comparaisons sur 26). Chaque lien peut être tracé jusqu'aux enregistrements qui le justifient. Le biais de popularité est compensé : aucune recommandation de F3 ne figure dans le top 100, et sa popularité médiane est 11 fois plus faible qu'une recommandation par popularité. Les index couvrants réduisent les temps de 39 à 63 %. Le placement par lecteur favorise les requêtes personnalisées et la résistance aux pannes, le placement par livre favorise l'exploration. Pour aller plus loin, il faudrait précalculer les voisins de chaque livre en les mettant à jour avec F6, utiliser de vrais replica sets, et mener E3 sur davantage de lecteurs.